

WEB COMPONENTS II

Comunicación entre componentes y la librería Lit

Apuntes revisados y ampliados

Documento elaborado a partir del fastbook original de Miguel Ángel Muñoz

Contenido

1. Comunicación entre componentes	3
1.1. Comunicación mediante atributos	3
1.2. Comunicación mediante eventos	3
1.3. Event bubbling	3
1.4. Contexto global y gestores de estado	3
2. Lit	4
2.1. Estilos y propiedades	5
2.2. Render y ciclo de vida	5
3. Qué hemos aprendido	6

1. Comunicación entre componentes

Una vez que se sabe crear un web component, la siguiente pregunta es casi obligada: ¿cómo hago que dos componentes se hablen entre sí? De eso depende, en la práctica, la reactividad, los cambios de estado y buena parte de la lógica de cualquier aplicación construida con componentes.

1.1. Comunicación mediante atributos

El caso más sencillo es la comunicación **Parent-to-Child**: el componente padre modifica un atributo del componente hijo, y este, si está observando ese atributo, reacciona en consecuencia. Por ejemplo, un `<form-envio>` (padre) podría fijar el atributo valor de un `<campo-input>` (hijo) para autorellenarlo tras una acción del usuario.

1.2. Comunicación mediante eventos

Para el sentido contrario —que el hijo avise al padre— la herramienta natural son los **eventos personalizados**. Crear uno es tan sencillo como instanciar un `CustomEvent` con un nombre y, opcionalmente, información adicional en detail:

```
const evento = new CustomEvent('producto-agregado', {
  bubbles: true,
  composed: true,
  detail: { id: 42 }
});
this.dispatchEvent(evento);
```

Cuatro opciones controlan su comportamiento: **bubbles** decide si el evento se propagará hacia los elementos ancestros; **cancelable** permite que se pueda cancelar con `event.preventDefault()`; **composed** indica si el evento debe poder atravesar la frontera del Shadow DOM; y **detail** transporta la información adicional que se quiera adjuntar. Por defecto, las cuatro valen false.

1.3. Event bubbling

El **event bubbling** es uno de los mecanismos más potentes —y más confusos al principio— de JavaScript: cuando un evento con `bubbles: true` ocurre en un elemento, va "burbujeando" hacia arriba por el árbol del DOM, permitiendo que cualquier ancestro lo escuche con `addEventListener`.

Dos herramientas resultan imprescindibles al trabajar con bubbling: `event.target`, que identifica el elemento donde se originó realmente el evento, y `event.stopPropagation()`, que detiene la propagación cuando no interesa que suba más allá de un punto concreto.

★ Regla clave

- Si un elemento vive dentro de un Shadow DOM y necesitas llegar a su padre real (el shadow host), usa `elemento.getRootNode().host` en lugar de `elemento.parentNode`.

1.4. Contexto global y gestores de estado

El objeto **window** actúa como contexto global y puede usarse —con mucho cuidado— para compartir información accesible desde cualquier parte del sitio. El problema es que, por sí solo, no es **reactivo**: si algo cambia en ese contexto, nadie se entera automáticamente salvo que se combine con algún patrón de notificación.

Ahí es donde entra un **gestor de estados**: una forma organizada de modificar, consultar y reaccionar a los cambios del conjunto de valores que describen la situación de la aplicación en cada momento. Para aplicaciones sencillas puede bastar con el contexto global más algún patrón de notificación; para aplicaciones más complejas conviene recurrir a una herramienta especializada como **Redux**, capaz de registrar cada cambio de estado, tanto síncrono como asíncrono.

El **patrón observador** formaliza esa idea: un **Subject** guarda el estado y notifica a sus **Observers** cada vez que cambia. Una variante muy utilizada es **PubSub** (Publisher/Subscriber), donde un canal intermediario gestiona las suscripciones sin que el publicador y los suscriptores lleguen a conocerse entre sí. Angular, por ejemplo, se apoya en el patrón observador (con su reciente Angular Signals como envoltorio simplificado), y RxJS es la librería de referencia para implementarlo en frontend.

★ Lo más importante

- ▶ Padre → hijo: atributos. Hijo → padre: eventos personalizados con bubbling.
- ▶ `stopPropagation()` corta la propagación; `getRootNode().host` cruza la frontera del Shadow DOM.
- ▶ Para estado global complejo, usa un gestor de estados o el patrón observador/PubSub en lugar del contexto global a pelo.

2. Lit

Lit es una librería —no un framework— que simplifica la creación de web components nativos. La diferencia con Angular, React o Vue es importante: el resultado final de usar Lit sigue siendo un componente nativo perfectamente funcional en cualquier entorno, sin quedar atado a ningún framework.

```

npm i lit

import { LitElement, html, css } from 'lit';

class MiContador extends LitElement {
  static properties = { contador: { type: Number } };
  static styles = css`button { padding: 8px 16px; }`;

  constructor() {
    super();
    this.contador = 0;
  }

  render() {
    return html`
      <button @click=${() => this.contador++}>
        Valor: ${this.contador}
      </button>
    `;
  }
}

customElements.define('mi-contador', MiContador);

```

2.1. Estilos y propiedades

La utilidad **css** permite definir estilos sin necesidad de envolverlos en una etiqueta `<style>`, asignándolos a la propiedad estática **static styles**. Por seguridad, Lit no permite inyectar variables directamente dentro de `css`: solo acepta constantes previamente marcadas como de tipo `css`, evitando así inyección de código.

Las propiedades reactivas se declaran con `static properties`, con varias opciones de configuración:

- type** — el tipo de dato esperado (por defecto, `String`).
- attribute** — si la propiedad se expone como atributo HTML.
- reflect** — si el valor de la propiedad se refleja también en el atributo.
- state** — marca la propiedad como interna (privada), sin atributo asociado.
- hasChanged** — función personalizada para decidir si ha habido un cambio real.

2.2. Render y ciclo de vida

El método **render()**, apoyado en la utilidad **html**, define la plantilla del componente. Debe ser una función pura: no debe modificar propiedades ni provocar efectos secundarios, y para los mismos valores de entrada debe devolver siempre el mismo resultado.

El ciclo de actualización de una propiedad sigue un orden muy concreto: `shouldUpdate` decide si procede actualizar; `willUpdate` calcula valores necesarios antes de renderizar; `update` refleja las propiedades en sus atributos y llama a `render`; y, tras el primer renderizado, `firstUpdated` y `updated` permiten reaccionar una vez que el componente ya está actualizado en pantalla.

Los eventos se gestionan en las plantillas de Lit con la sintaxis `@evento=${...}`, como se ve en el ejemplo del contador: mucho más directo que llamar a `addEventListener` a mano.

★ Lo más importante

- ▶ Lit genera componentes nativos reales, sin atarte a un framework.
- ▶ `static properties` define qué propiedades son reactivas y cómo se exponen.
- ▶ El método `render()` debe ser puro: sin efectos secundarios ni cambios de estado.

3. Qué hemos aprendido

En este documento hemos visto cómo se comunican los web components entre sí —mediante atributos de padre a hijo, y mediante eventos personalizados con bubbling de hijo a padre—, cómo compartir estado más allá de esa relación directa (contexto global, gestores de estado, patrón observador y PubSub) y cómo Lit simplifica notablemente la creación de componentes nativos sin perder ninguna de sus ventajas.

La mejor forma de asimilar todo esto es, de nuevo, la práctica: construir un pequeño conjunto de componentes que se comuniquen entre sí es el ejercicio que más rápido consolida estos conceptos.

¡Enhorabuena! Documento completado